

Question (2)

```
import numpy as np
# algorithm help from wikipedia
# m is order of derivative
# n is order of accuracy
def taylor(m,n):
    size_1 = 2*np.round((m+1)/2) - 1 + n
    p = np.int((size_1 - 1)*0.5)
    size_1 = np.int(size_1)
    A = np.zeros(shape=(size_1,size_1))
    for i in range(size_1):
        for j in range(size_1):
            A[i,j] = (-p + j)**i
    b = np.zeros(size_1)
    b[m] = np.math.factorial(m)
    a_coeff=np.dot(np.linalg.inv(A),b)
    return a_coeff

>>> 840*taylor(1,8)
array([ 3.00000000e+00, -3.20000000e+01,  1.68000000e+02, -6.72000000e+02,
        1.88542392e-11,  6.72000000e+02, -1.68000000e+02,  3.20000000e+01,
       -3.00000000e+00])

# the output is as follows a(-n) a(-n+1) a(-n+2) ..... a(n-1) a(n)

# here the value of a_0 is not exactly zero but very close to zero
# this error exist because of matrix inverse and round off errors

# other examples are as follows

>>> 12*taylor(1,4)
array([ 1.00000000e+00, -8.00000000e+00,  2.85485921e-15,  8.00000000e+00,
       -1.00000000e+00])

>>> 60*taylor(1,6)
array([-1.00000000e+00,  9.00000000e+00, -4.50000000e+01, -8.48249007e-14,
        4.50000000e+01, -9.00000000e+00,  1.00000000e+00])
```

Question (3 a)

```

import numpy as np
import matplotlib.pyplot as plt

tos = 1000
x=np.arange(0.0,1100,0.5)
d_x = 0.5
a0 = 0.0
a1 = 0.77088238051822552
a2 = -0.166705904414580469
a3 = 0.02084314277031176
a_1 = -a1
a_2 = -a2
a_3 = -a3

d_t = 0.05
b0 = 2.3025580888383
b1 = -2.4910075998482
b2 = 1.5743409331815
b3 = -0.3858914221716

def ini_con(x):
    ind_1 = (np.where((x<20.1) & (x>19.9)))[0][0]
    ind_2 = (np.where((x<30.1) & (x>29.9)))[0][0]
    u = 0.0*np.asarray(x)
    u[ind_1:ind_2]=np.sin(0.2*np.pi*(x[ind_1:ind_2]-20))
    return u

def act_sol(x,tos):
    ind_1 = (np.where((x<20.1) & (x>19.9)))[0][0] + np.int(tos)*2
    ind_2 = (np.where((x<30.1) & (x>29.9)))[0][0] + np.int(tos)*2
    u = 0.0*np.asarray(x)
    u[ind_1:ind_2]=np.sin(0.2*np.pi*(x[ind_1:ind_2]-20))
    return u

u = ini_con(x)

xi = np.arange(3,np.shape(x)[0]-3)

plt.figure(1)
#plt.plot(x,u,'g',label='Initial state')
#plt.show()

du_dt = np.array([0.0*np.asarray(xi) for i in range(4)])

tim = 0.0
while tim<=tos:
    du_dx = (a0*u[xi] + a1*u[xi+1] + a2*u[xi+2] + a3*u[xi+3])
    du_dx += (a_1*u[xi-1] + a_2*u[xi-2] + a_3*u[xi-3])
    du_dx *= (1/d_x)

    du_dt[0]= - du_dx
    u[xi] = u[xi] + d_t * (b0*du_dt[0] + b1*du_dt[1] + b2*du_dt[2] + b3*du_dt[3] )

    du_dt[1:4] = du_dt[0:3]

    tim += d_t

#plt.plot(x,u,'r',label='final state')
#plt.plot(x,act_sol(x,tos),'b',label='actual solution')

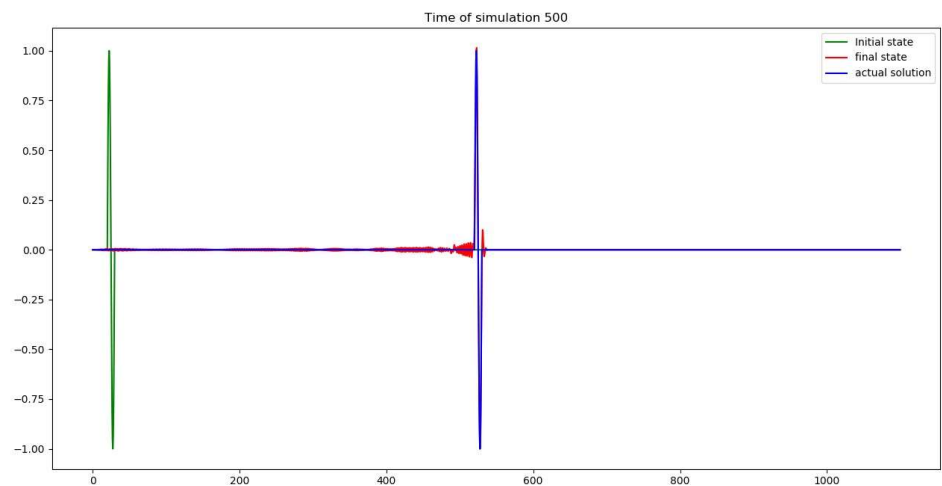
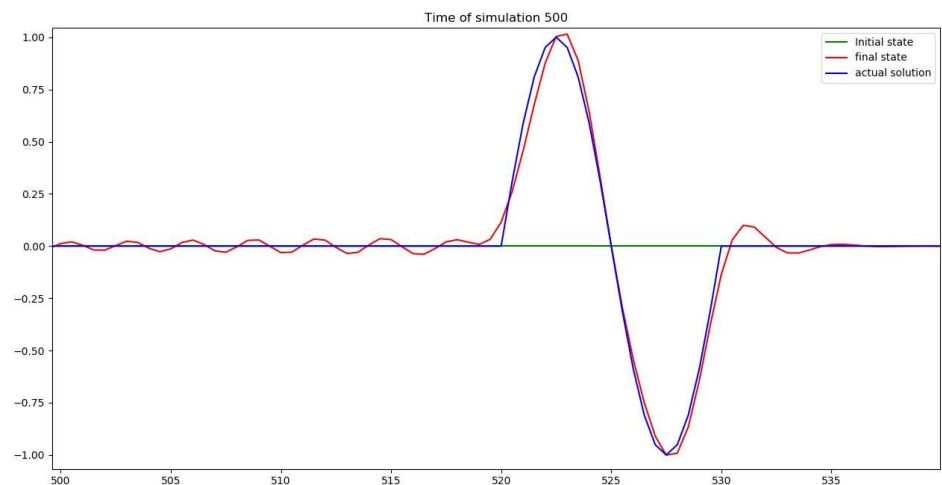
```

```
plt.plot(x,np.abs(act_sol(x,tos)-u),'b')
plt.title('Time of simulation '+str(tos))
plt.xlabel('Time')
plt.ylabel('Error')
plt.show()
```

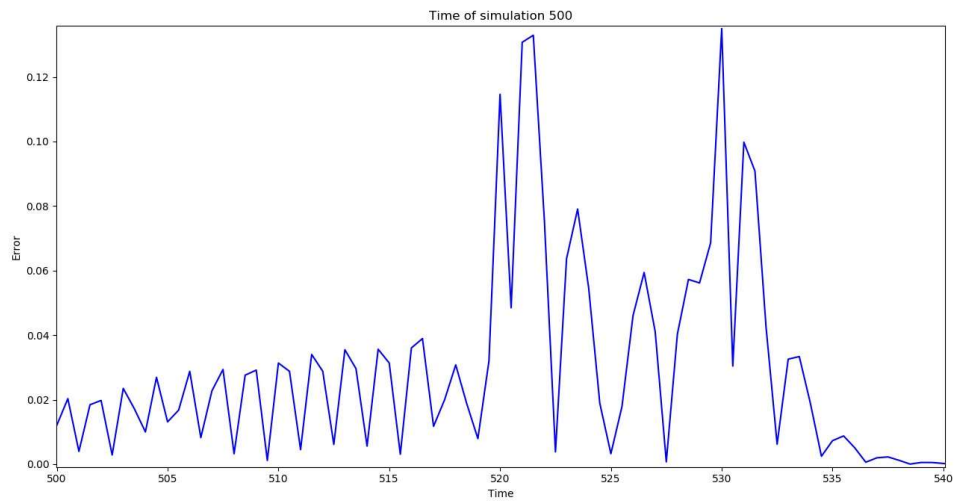
Question (3b)

Q3)

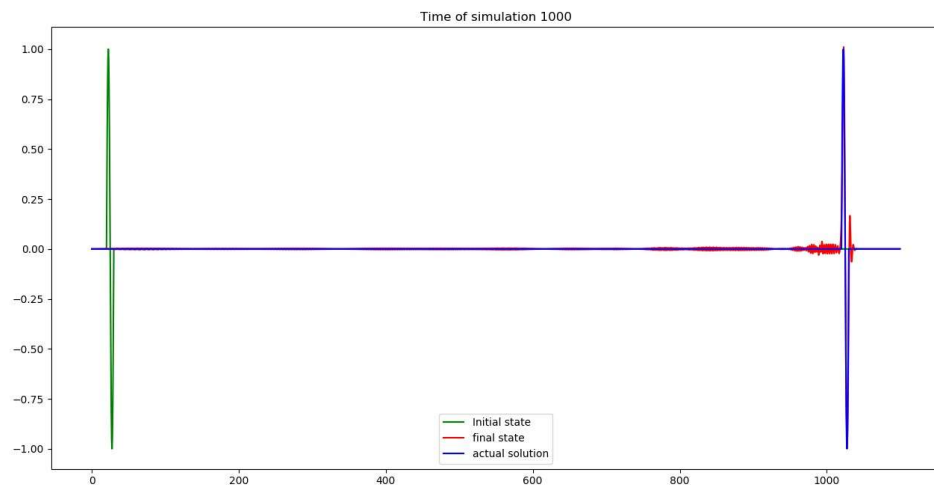
a) At time 500 sec

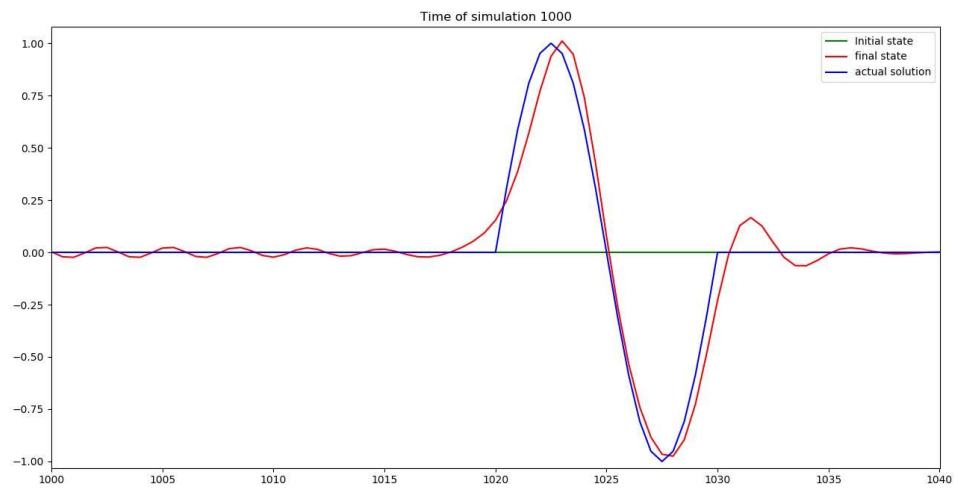


b) 500 sec error



c) At time 1000 sec





d) 1000 sec error

